

شرح شامل: Response Engine — الصفحتان itsm/playbooks/ و itsm/automations/

أولاً: ما العلاقة بين الصفحتين؟

الصفحتان مترابطتان بشكل عضوي:

الصفحة	ما تفعله	التشبيه
itsm/playbooks/	تُنشئ وتُدير "كتب التشغيل"	الوصفة الطبية (ماذا نفعل؟)
itsm/automations/	تُنشئ قواعد تُطلق هذه الكتب تلقائياً	متى نُعطي الوصفة؟

القاعدة الذهبية: الـ Automation Rule تُشير لـ Playbook. بدون Playbook لا تعمل القاعدة.

ثانياً: صفحة itsm/playbooks/ — بالتفصيل الكامل

ما هو الـ Playbook أصلاً؟

الـ Playbook = قائمة أوامر مرتبة يُرسلها السيرفر للـ Agent تنفيذاً لخطة استجابة محددة.

Playbook: "Ransomware Containment"

- Command 1: isolate_network → قطع الشبكة فوراً
- Command 2: process_tree_snapshot → التقط شجرة العمليات
- Command 3: memory_dump → احتفظ بنسخة من الذاكرة
- Command 4: persistence_scan → Persistence افحص نقاط الـ

مما تتكون الصفحة؟

1. Header — رأس الصفحة

عند فتح الصفحة مباشرة:

- عنوان: Incident Response Playbooks

- زر Create Playbook (أزرق)

عند القدوم من صفحة تنبيه (عبر navigation state):

- يظهر شريط **Active Alert Context** أخضر اللون
- يعني: "أنت قادم من تنبيه معين، وستنفذ الـ Playbook على الجهاز المرتبط به"
- يُملأ **Agent ID** تلقائياً من بيانات التنبيه

2. AlertContextPanel — لوحة سياق التنبيه

تظهر فقط عند القدوم من تنبيه. تعرض:

- معرف التنبيه
- الخطورة (severity)
- اسم الجهاز
- اسم القاعدة التي أطلقت التنبيه

الهدف: يذكر المحلل بالسياق الذي جاء منه قبل أن ينفذ أي شيء.

3. UserAssistant — المساعد الذكي

مكوّن يعرض اقتراحات ذكية بناءً على سياق التنبيه:

- لو التنبيه severity=critical + category=containment → يقترح Playbooks الاحتواء
- لو ruleName يحتوي "malware" → يقترح Playbooks التحقيق
- لو ruleName يحتوي "ransomware" → يقترح Playbooks Ransomware

من أين تأتي هذه الاقتراحات؟ من الـ Frontend مباشرة (ليس من الـ Backend). الكود يُصَفّي الـ Playbooks المحمّلة بناءً على شروط بسيطة:

```
// PlaybooksPage.tsx - سطر 147-157
if (severity === 'critical' && playbook.category === 'containment') return true;
if (ruleName.includes('malware') && playbook.category === 'investigation') return true;
if (ruleName.includes('usb') && playbook.name.toLowerCase().includes('usb')) return true;
if (ruleName.includes('ransomware') && playbook.name.toLowerCase().includes('ransomware')) return true;
```

4. Recommended Playbooks — الاقتراحات

قسم يظهر فقط عند وجود **Alert Context** ويعرض الـ Playbooks المقترحة بناءً على الفترة أعلاه.

لكل Playbook مقترح: زر **Run Playbook** مباشر.

5. Available Playbooks — كل الـ Playbooks

القائمة الكاملة لكل الـ Playbooks المخزنة في قاعدة البيانات.

لكل Playbook يُعرض:

- الاسم والوصف
- الـ **Category** (containment / investigation / remediation / validation) — بلون مختلف لكل نوع
- عدد الأوامر فيه
- تقنيات MITRE المرتبطة به
- تاريخ الإنشاء
- 3 أزرار: **Execute** | **View Details** | **Delete**

6. Execute Modal — نافذة التنفيذ اليدوي

عند الضغط على **Execute**، تظهر نافذة منبثقة تحتوي:

أ. اختيار الـ **Agent** المستهدف:

- قائمة منسدلة بكل الـ Agents المسجلة
- لو أُتيبت من تنبيه → يُملأ تلقائياً بـ agentId من بيانات التنبيه

ب. معاملات الأوامر (**Parameters**):

لكل أمر في الـ Playbook يحتاج معاملات، يظهر حقل إدخال:

نوع الأمر	الحقل المطلوب	مثال
terminate_process	اسم العملية أو PID	vssadmin.exe أو PID:1234
quarantine_file	مسار الملف	C:\Windows\Temp\malware.exe
run_cmd	الأمر	...\reg query HKLM
collect_logs	قنوات الـ Log (checklist)	System, Security, Sysmon
scan_file	المسار المراد فحصه	C:\Users\victim\Downloads
collect_forensics	قنوات + max events	Security, 500 events
filesystem_timeline	النافذة الزمنية بالساعات	24

ذكاء الـ **Auto-fill**:

لو جاء المحلل من تنبيه فيه بيانات الحدث (event_data):

- terminate_process → يُملأ تلقائياً من event_data.Image أو event_data.ProcessId

- quarantine_file → يُملأ من event_data.TargetFilename أو event_data.Image
- scan_file → يُملأ من event_data.TargetFilename

ج. Execution Sequence — تسلسل التنفيذ:

قبل الضغط Execute، يعرض قائمة مرئية بكل الخطوات بالترتيب.

د. Live Progress — التقدم المباشر:

أثناء التنفيذ:

- الخطوة الحالية تضيء بلون indigo + animation
- الخطوات المكتملة تُعلَّم بعلامة صح خضراء
- الخطوة الفاشلة تُعلَّم بـ X حمراء
- لو فشلت خطوة → يتوقف التنفيذ ويُعرض رسالة الخطأ

7. كيف يتم التنفيذ فعلياً؟

"Execute Playbook" المحلل يضغط

↓

هل ملأ الحقول المطلوبة؟ Agent يتحقق: هل اختار Frontend

↓

for (كل command في Playbook) {

Agent موسعة في الـ whitelist مهم! يفتح ← params.from_playbook = "true"

```
await agentsApi.executeCommand(agentId, {
  command_type: mappedType,
  parameters: params,
  timeout: cmd.timeout
})
```

للتأثير البصري ← await delay(800ms)

}

↓

يستقبل الطلب Backend (Connection Manager)

↓

commands (status=pending) في جدول Command record يُنشئ

↓

Agent للـ gRPC stream يُرسل عبر

↓

(CommandResult) يُنفذ ويُرسل النتيجة Agent

ملاحظة مهمة: التنفيذ من هذه الصفحة يحدث أمر بأمر وليس عبر AutomationService.ExecutePlaybookForAlert . كل أمر

يُرسل مستقل عبر agentsApi.executeCommand . الـ Frontend هو من يتحكم في التسلسل.

8. Create Playbook Modal — إنشاء Playbook جديد

يطلب فقط:

- الاسم
- الوصف
- الـ Category

ويُنشئ Playbook بأمر واحد افتراضي (`isolate_network`). بعد الإنشاء، يجب تعديله لإضافة الأوامر الباقية.

ثالثاً: صفحة `itsm/automations/` — بالتفصيل الكامل

ما هي قاعدة الأتمتة (Automation Rule)؟

هي تعليمة دائمة تقول للسيرفر:

"لو وصل تنبيه يطابق هذه الشروط → نفذ هذا الـ Playbook (تلقائياً أو اقترحه)"

مما تتكون الصفحة؟

1. Header

- عنوان: **Automation Rules**
- زر **Create Rule**
- شريط **Active Alert Context** لو جاء من تنبيه

2. AlertContextPanel + UserAssistant

نفس المكوّنين في صفحة الـ Playbooks، لكن هنا UserAssistant يقترح إنشاء قاعدة جديدة بدلاً من تنفيذ Playbook.

3. قائمة القواعد — كل ما يُعرض لكل قاعدة

لكل قاعدة موجودة:

الحقل	المصدر	ما يعنيه
الاسم	<code>rule.name</code>	اسم القاعدة
Priority badge	<code>rule.priority</code>	Critical / 3-5=High / 6-8=Medium / 1-2=Low / 9+=Low

الحقل	المصدر	ما يعنيه
Matches Active Alert"" badge	rule.matchesCurrentAlert	تُضيء لو القاعدة تطابق التنبيه الحالي
Success Rate	rule.successRate	نسبة نجاح التنفيذ التاريخية (من automation_metrics)
Cooldown	rule.cooldownMinutes	فترة الراحة بين التنفيذات
Linked Playbook	rule.playbookId	اسم الـ Playbook المرتبط
Last Run	rule.lastExecution	آخر مرة نُفِّذت (أو "Never executed")
Trigger Condition	rule.triggerConditions	الشروط كـ JSON أو نص

4. أضرار كل قاعدة

الزر	الوظيفة	ما يحدث في الـ Backend
Enable/Disable	تفعيل أو تعطيل القاعدة	PUT /automation/rules/:id يُحدّث enabled في DB
Configure	تعديل إعدادات القاعدة	يفتح نفس Modal الإنشاء بالبيانات الحالية
Delete	حذف القاعدة	DELETE /automation/rules/:id

التفعيل/التعطيل يعمل بـ Optimistic UI:

- التغيير يظهر فوراً في الـ UI قبل انتهاء الطلب
- لو فشل الطلب → يعود للحالة السابقة تلقائياً

5. Create/Edit Rule Modal — إنشاء أو تعديل قاعدة

يحتوي على 4 حقول:

أ. Rule Name

مثل: "Contain Ransomware Behavior"

لو جاء من تنبيه → يُملأ تلقائياً:

"Response Rule for: Suspicious Ransomware Activity"

ب. Trigger Condition

نص حر (SQL-like أو Sigma-like):

```
RuleName == "Suspicious File Write" && RiskScore > 80
```

لو جاء من تنبيه → يُملأ تلقائياً:

```
RuleName == "Suspicious Ransomware Activity" && Severity == "critical"
```

ملاحظة: هذا الشرط يُحفظ كـ { condition: "النص" } في جدول automation_rules.trigger_conditions كـ JSONB. السيرفر في evaluateAdvancedConditions يقرأه ويُقيّمه، لكن التقييم الحالي يعتمد على severity [] و rule_patterns [] وليس على هذا الشرط النصي مباشرة.

ج. Target Playbook

قائمة منسدلة بكل الـ Playbooks المتاحة في DB. المحلل يختار أي Playbook سيُشغّله هذه القاعدة.

د. Enable Auto-Execution

checkbox واحد يُحدد:

- **مُفَعَّل** → autoExecute=true → القاعدة تُنفَّذ الـ Playbook تلقائياً بدون موافقة بشرية
- **غير مُفَعَّل** → autoExecute=false → القاعدة تقترح الـ Playbook فقط، والمحلل يوافق يدوياً

رابعاً: أنواع الاستجابة — كاملة ومفصّلة

النوع 1: الاستجابة اليدوية (Manual Response)

متى تحدث؟ عندما يتدخل المحلل بنفسه.

السيناريوهات:

A-1: تنفيذ Playbook من صفحة itsm/playbooks/

itsm/playbooks /المحلل → يفتح

→ يختار Playbook

→ يختار Agent

→ يملأ المعاملات

→ يضغط Execute

→ agentsApi.executeCommand يرسل أوامر واحدة بوحدة عبر Frontend

(commands تُسجّل فقط في) جدول playbook_executions لا تُسجّل في

B-1: عزل الجهاز يدوياً من Endpoint Detail

المحلل → يفتح صفحة الجهاز

→ "Isolate" يضغط

→ REST API: POST /agents/:id/commands

{ command_type: "isolate_network" }

→ gRPC → Agent → يعزل نفسه

→ Agent (CommandResult) يُرسل نجاح

→ server.go: OnIsolationSucceeded() ← يُطلق playbook.Engine

C: Memory Dump (يتطلب تأكيد صريح)

POST /agents/:id/collect-memory

Body: { "confirm": true } ← body مطلوب صراحةً في

يُرفض الطلب فوراً → confirm != true لو

D: Mark False Positive-1

POST /agents/:id/incident/false-positive

→ playbook_runs.status = 'false_positive' يُحدّث

→ Audit Log (fireAudit) يُسجّل في

E: Escalate Incident-1

POST /agents/:id/incident/escalate

→ "escalated": true في playbook_runs.summary (JSONB merge) يُضيف

→ Audit Log يُسجّل في

توثيق الاستجابة اليدوية:

أين تُحفظ	آلية التوثيق
UUID المحلل المنفّذ	PlaybookExecution.CreatedBy
user + action + target + timestamp	Audit Log (fireAudit)
ID من أصدر الأمر	commands.issued_by
يحتوي escalated=true إذا رُفّع	playbook_runs.summary

النوع 2: الاستجابة التلقائية — AutomationService

متى تحدث؟ عند وصول تنبيه يطابق قاعدة `autoExecute=true`.

المسار الكامل:

Detection Engine ينتج Alert



ProcessAlert(alert) يُستدعى



Step 1: DB → جلب كل `automation_rules` WHERE `enabled=true`



Step 2: MLOptimizer → `successRate` ترتيب حسب (آخر 30 يوم `automation_metrics` من)



Step 3: Rule لكل:

- Gate 1: `isInCooldown?` (in-memory map) → ارفض لو في فترة راحة
- Gate 2: `successRate < 0.5?` → ارفض لو أداء سيئ
- Gate 3: `evaluateAdvancedConditions()?` → ارفض لو الشروط لا تطابق



Step 4: Gates اجتاز الـ `AutoExecute=true` لو:

- `go executePlaybookAsync()` [goroutine مستقلة]
- إنشاء `PlaybookExecution` (`status=running`, `CreatedBy=nil`)
- جلب `Playbook.Commands` من DB
- `CommandService.ExecutePlaybookCommand()`
 - `inject: from_playbook=true`
 - إنشاء `Command` في DB
 - `WaitForCommandResult` (polling كل 2 ثانية)
- تحديث `PlaybookExecution.status`
- `updateRuleMetrics` (`SuccessRate = 0.8×old + 0.2×new`)
- `setCooldown(ruleID)`



Step 5: `AutoExecute` بغض النظر عن) دائماً:

- `generateIntelligentSuggestions()`
- `playbook_suggestions` تُخزن في
- Dashboard تظهر في

متى يُفشل كل صمام أمان؟

الصمام	متى يرفض	لماذا؟
Cooldown	مَرَّ أقل من <code>cooldownMinutes</code>	منع إغراق الـ Agent بأوامر متكررة

الصمام	متى يرفض	لماذا؟
SuccessRate < 50%	القاعدة فشلت تاريخياً	أوامرها غير فعالة أو خاطئة
[]severity	التنبية ليس في قائمة الخطورات	تجنب الاستجابة لتنبیهات منخفضة الأهمية
[]rule_patterns	RuleName لا يحتوي أي pattern	دقة في مطابقة نوع التهديد
confidence_threshold	confidence < threshold	لا تستجب لتنبیهات منخفضة الثقة

النوع 3: الاستجابة الفورية — playbook.Engine

متى تحدث؟ تلقائياً 100% بعد نجاح عزل الجهاز.

المسار:

```

يُرسَل نجاح عزل الشبكة Agent
↓
server.go → SendCommandResult()
  cmdType == "isolate_network"
↓
updateAgentIsolation(true) ← تحديث DB
playbookEngine.OnIsolationSucceeded(agentID)
↓
Engine running[agentID]?
  (منع التكرار) skip → نعم
  لا → running[agentID] = true
↓
runPlaybook("default_post_isolation")
  → CreateRun في DB (status=running)
  → خطوات بالترتيب 7:
    step: لكل
      IsOnline(agentID)? → offline → skipped
      CreateStep في DB
      Build proto Command
      AgentRegistry.Send(agentID, cmd) ← مباشر عبر gRPC
      sleep(200ms)
    steps يستقبل النتائج ويُحدَّث ← OnCommandResult()
    → FinishRun: completed / partial / failed
↓
running[agentID] = false ← يسمح بتشغيل آخر مستقبلاً

```

خامساً: جدول مقارنة الأنواع الثلاثة

المعيار	Manual (Playbooks Page)	Auto (AutomationService)	Post-Isolation (Engine)
من يُقرر	المحلل البشري	النظام تلقائياً	النظام تلقائياً
المُحفّز	ضغط زر من Dashboard	وصول تنبيه يطابق قاعدة	نجاح عزل الجهاز
الـ Playbooks	Dynamic (DB)	Dynamic (DB)	Embedded YAML
يُسجّل في	commands table	playbook_executions + commands	playbook_runs + playbook_steps
من يُرسل للـ Agent	agentsApi (Frontend)	CommandService (Backend)	AgentRegistry.Send (Backend)
التوثيق	CreatedBy = userID	CreatedBy = nil	playbook_runs.trigger
يمكن تعديل الـ Playbook	نعم (من Dashboard)	نعم (من Dashboard)	لا (يحتاج rebuild)
صمامات أمان	لا	3 صمامات	duplicate guard

سادساً: ما لا تفعله الصفحتان حالياً

الميزة	الحالة
تنفيذ Playbook عبر AutomationService.ExecutePlaybookForAlert	غير مُستخدَم من الـ Frontend
عرض playbook_suggestions من الـ Backend	الاقتراحات تحدث في Frontend مباشرة
استقبال نتائج التنفيذ real-time عبر WebSocket	لا يوجد
تعديل أوامر الـ Playbook من الـ UI	لا يوجد (يحتاج API جديد)
عرض تاريخ تنفيذ القواعد (automation_metrics)	لا يوجد